

Guía Completa de Uso de Git y GitHub

1. Introducción al Control de Versiones

El control de versiones es el sistema que registra los cambios realizados en uno o varios archivos a lo largo del tiempo, de modo que podamos recuperar versiones específicas más adelante. En el proyecto **EuroCoinCollector**, utilizamos **Git** como herramienta de control de versiones local y **GitHub** como plataforma de alojamiento remoto en la nube.

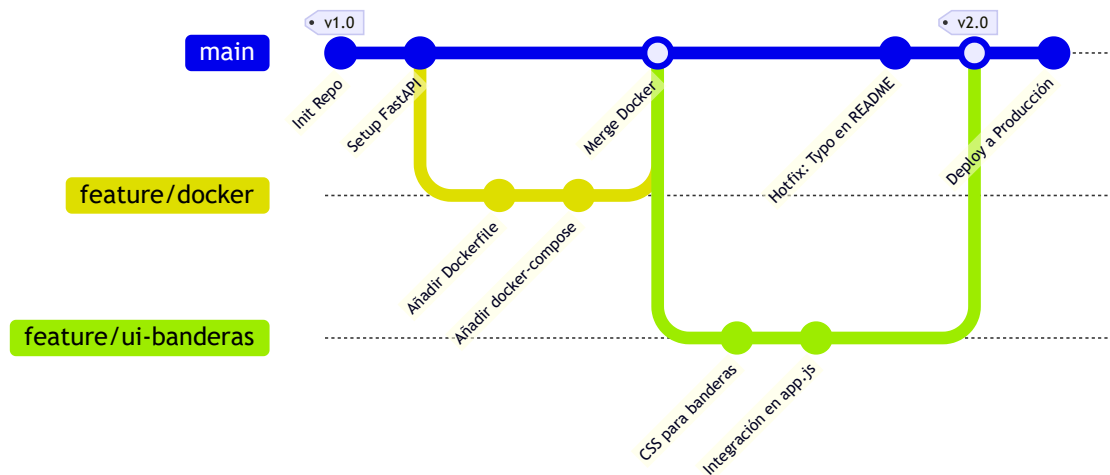
¿Por qué lo hemos realizado de esta manera?

Utilizar Git y GitHub nos proporciona una red de seguridad inmensa. Si rompemos el código, podemos volver a una versión anterior en segundos. Además, permite a múltiples personas (o a uno mismo desde distintos ordenadores) trabajar en paralelo sin sobrescribir el progreso del otro, manteniendo un registro de "quién hizo qué y por qué".

2. Flujo de Trabajo (Workflow) Recomendado

En este proyecto, seguimos un modelo simplificado basado en **Feature Branches** (Ramas de Características). Nunca trabajamos directamente sobre la rama principal (`main`), sino que creamos una rama paralela, desarrollamos la funcionalidad, la probamos y finalmente la unimos (`merge`).

Diagrama de Flujo de Ramas



Explicación del Flujo

1. **main** : Es la fuente de la verdad. El código aquí siempre debe funcionar y estar listo para producción.
2. **feature/*** : Para cada nueva funcionalidad (ej. **feature/ui-banderas**), creamos una rama. Aquí experimentamos y construimos.
3. **Merge**: Cuando la funcionalidad está terminada y probada, la integramos de vuelta a **main** .

3. Comandos Diarios Esenciales

A continuación se explican en detalle los comandos que utilizamos día a día en el desarrollo.

3.1. Obtener actualizaciones

Antes de empezar a programar, siempre debemos descargar los últimos cambios de GitHub.

```
# Descarga los cambios de la nube pero no los fusiona aún  
git fetch origin
```

```
# Descarga los cambios y los fusiona directamente en tu rama actual  
git pull origin main
```

¿Por qué? Para evitar conflictos. Si alguien más subió código, debes tenerlo en tu ordenador antes de escribir código nuevo.

3.2. Crear una nueva rama

Vas a añadir el modo oscuro. No toques `main`.

```
# Crea una rama llamada 'feature/dark-mode' y te mueve a ella
git checkout -b feature/dark-mode
```

3.3. Guardar el trabajo (Commit)

Has terminado de escribir el CSS. Es hora de guardar el paquete de cambios.

```
# Añade todos los archivos modificados a la "zona de preparación" (Staging)
git add .

# O, mejor aún, añade solo un archivo específico:
git add frontend/style.css

# Crea el "paquete" con un mensaje descriptivo
git commit -m "feat(ui): Añadir variables de color para dark mode"
```

¿Por qué hacemos *staging* (`add`) antes del `commit` ? Porque quizás modificaste 5 archivos, pero solo quieres agrupar 2 de ellos en un commit específico. Git te da control granular.

3.4. Subir a la nube (Push)

Tu trabajo está a salvo en tu ordenador local, pero GitHub aún no lo tiene.

```
# Sube tu rama a GitHub
git push origin feature/dark-mode
```

4. Buenas Prácticas y Convenciones

Para mantener un historial limpio y profesional, el proyecto se adhiere a la especificación **Conventional Commits**.

Estructura de un Commit

`<tipo>(<alcance opcional>): <descripción breve>`

Tipos soportados:

- `feat` : Una nueva característica (ej. `feat(api): endpoint de exportación a CSV`)
- `fix` : Corrección de un bug (ej. `fix(ui): botón de eliminar no respondía en móviles`)
- `docs` : Cambios en la documentación (ej. `docs: actualizar diagrama de arquitectura`)
- `style` : Formato de código (espacios, punto y coma) que no altera la lógica.
- `refactor` : Reescribir código sin añadir funcionalidad ni arreglar bugs.
- `test` : Añadir o corregir pruebas.
- `chore` : Tareas de mantenimiento, actualización de librerías, etc.

¿Por qué lo hacemos así?

Al mirar el historial del proyecto con `git log`, es extremadamente fácil entender qué ha pasado de un vistazo. Además, herramientas automáticas pueden leer estos prefijos y generar un archivo `CHANGELOG.md` por nosotros.

5. Resolución de Conflictos (Conflict Resolution)

El temor más común al usar Git. Ocurre cuando dos personas editan la **misma línea del mismo archivo** y Git no sabe cuál versión debe conservar.

Paso a paso para resolver un conflicto:

1. Al hacer `git merge` o `git pull`, Git avisará: `CONFLICT (content): Merge conflict in app.js`.
2. Abre `app.js` en VSCode. Verás algo así: ```javascript <<<<<< HEAD
const color = "red"; ===== const color = "blue";`

```
| | | | | | | feature/otra-  
| | | | | | | rama ``
```

3. Debes borrar las líneas de marcadores (<<<< , ==== , >>>>) y dejar el código final como quieres que quede.
4. Una vez corregido, guarda el archivo y dile a Git que el conflicto está resuelto: `bash git add app.js git commit -m "Merge: resolución de conflicto en color"`

Archivo .gitignore

En nuestro repositorio verás un archivo llamado `.gitignore` . Aquí listamos carpetas y archivos que Git **jamás** debe vigilar. - `__pycache__/` - `venv/` o `.env` - Archivos `.pdf` de reportes generados.

¿Por qué? Nunca debemos subir contraseñas (`.env`), archivos compilados, ni dependencias inmensas (`node_modules/` o entornos virtuales de Python) a GitHub. Hacen que el repositorio pese gigabytes y comprometen la seguridad.

6. Conclusión de la Guía

Git no es un sistema de copias de seguridad estático, es una máquina del tiempo y una herramienta de colaboración. Al seguir este flujo de Ramas -> Add -> Commit descriptivo -> Push, aseguramos que el crecimiento de **EuroCoinCollector** sea ordenado, reversible y preparado para el trabajo en equipo a gran escala.